EasyTrainer 使用教程

目录

1. 快速上手
2. 界面与项目
3. 标注
4. 训练
5. 模型管理
6. 测试与评估报告
7. OCR
8. 日志

快速上手

从空软件到跑出第一个模型，最短路径是这六步。每一步的细节在后面各章。

1. **添加项目** —— 点左侧「+ 添加项目」，输个名字。
2. **添加数据集** —— 右键项目节点 →「添加数据集」。
3. **导入数据** —— 选中数据集，点顶部「导入」，分别选图像文件夹和标注文件夹。
4. **核对数据** —— 点「属性」，确认类别和数量没搞错。
5. **训练** —— 点「训练」，选任务类型和数据集，其余留默认，点「开始训练」。
6. **看结果** —— 训练完在「模型」里找到这条记录，点「测试」在验证集上跑一遍；要交付就导一份 PDF 报告。

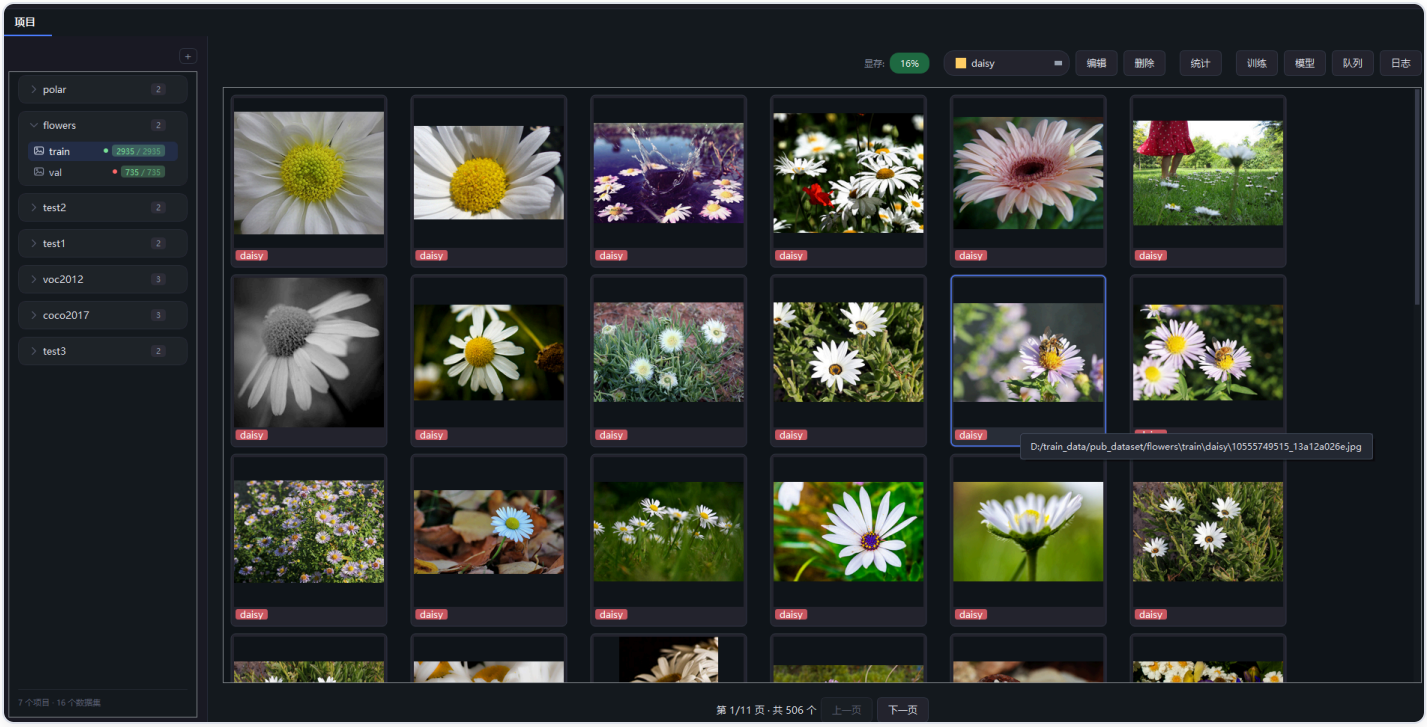
> 手上还没有标注数据的话，先看第 3 章「标注」。

1. 界面与项目

1.1 主界面布局

主界面分三块：左侧管项目，中间看图，工具栏在上方。

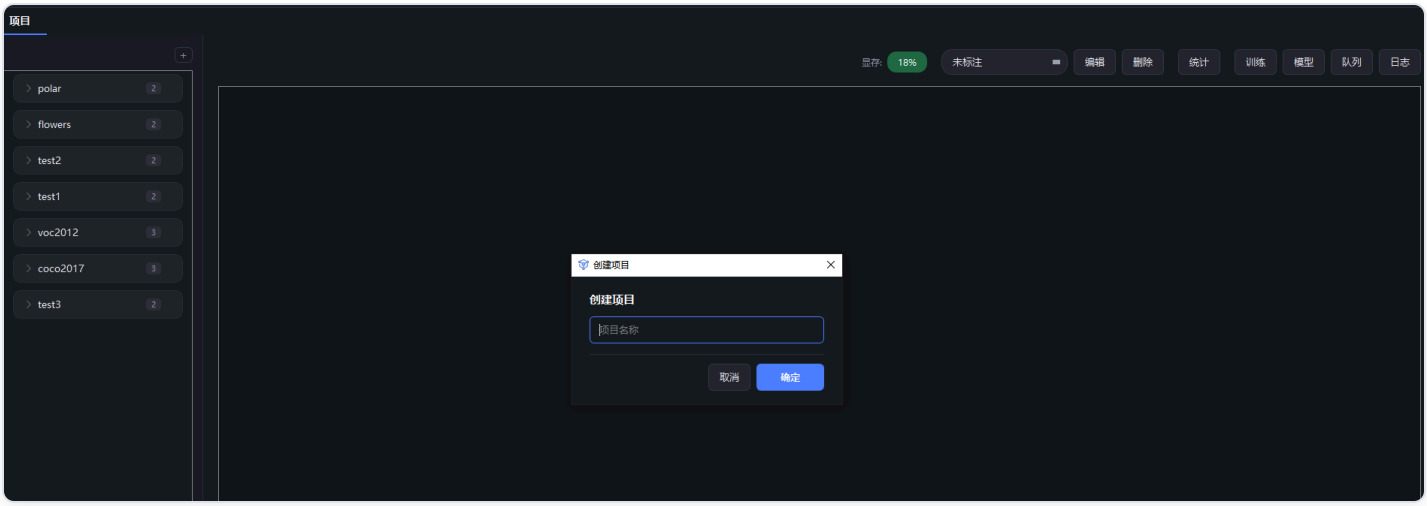
- **左侧项目树** —— 项目与数据集按层级排列。数据集后面是标注进度（如 `126/128`）；名字前的圆点表示它有没有载入内存，绿色已载入，红色未载入。
- **中间图像区** —— 缩略图分页浏览。底部居中一组翻页控件：两个箭头夹着页码，当前页数字高亮；鼠标停在页码上能看到当前筛选下共有多少张图。
- **顶部工具栏** —— 显示比例、标注筛选下拉框，以及编辑、删除、导入、导出、属性、训练、模型、日志等按钮。



EasyTrainer 主界面

1.2 项目与数据集

添加项目：点左侧「+ 添加项目」，输入名称，确定。项目是数据集的容器，每个数据集都必须属于某个项目。



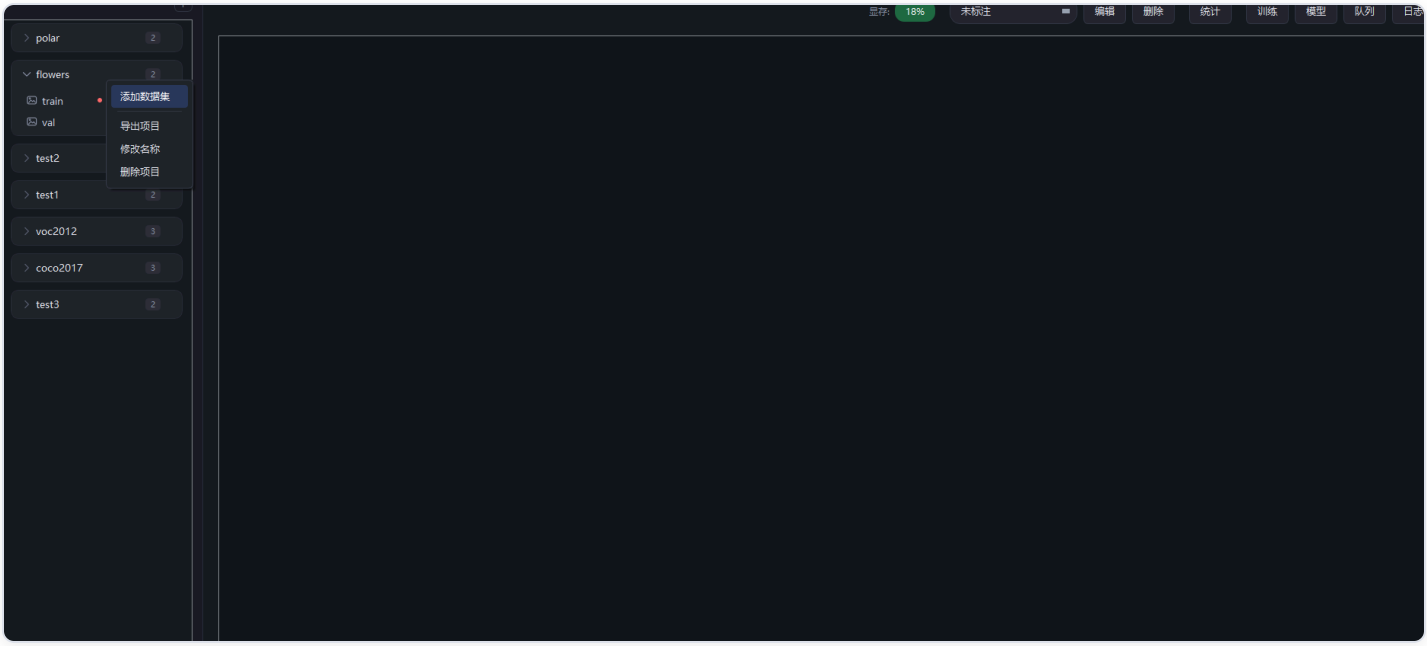
输入项目名称

右键**项目节点**：

- **添加数据集** —— 在项目下新建一个数据集
- **导出项目** —— 把整个项目的数据集与标注导出成指定格式
- **修改名称** —— 重命名项目
- **删除项目** —— 连同下面的数据集、标注、训练记录一起清除

右键**数据集节点**是对单个数据集的操作：导入 / 导出 / 移动 / 修改 / 删除。

双击数据集才会载入：重新扫描图像和标注、重建缓存，之后右侧就能浏览。已经载入过的（行内有状态圆点）再双击不会重复扫描。



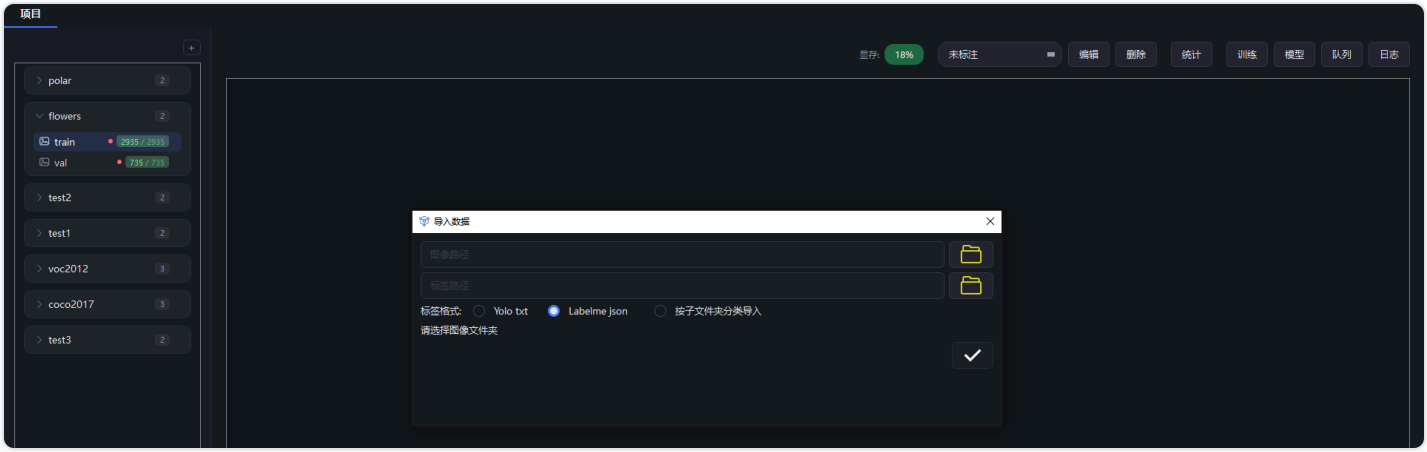
项目右键菜单

1.3 导入数据

选中数据集，点顶部「导入」，弹出导入对话框：

- **图像路径** —— 图片所在的文件夹
- **标签路径** —— 标注文件所在的文件夹
- **标签格式** —— Yolo txt / Labelme json / 按子文件夹分类导入

> **分类数据集**选「按子文件夹分类导入」：每个子文件夹名就是一个类别，**不需要标注文件。**
> OCR 用 Labelme json，文本框的标签固定为「文本」，导入后逐图打上文本标注标志；之后只有含文本标注的数据集会出现在 OCR 的训练集下拉里。



导入数据对话框

1.4 数据集属性

选中数据集，点顶部「属性」，能看到图像路径、标签路径，以及**标签分布柱状图** —— 每个类别有多少个标注框。哪一类样本偏少，一眼就能看出来。



数据集属性与标签分布

1.5 界面语言

工具栏最右侧是界面语言下拉框，九种语言任选：中文、English、繁體中文、日本語、한국어、Deutsch、Español、Français、Tiếng Việt。

选中后主界面立刻切换，已经打开的日志、训练队列窗口一起刷新；还没打开的对话框下次打开就是新语言。选择会被记住，下次启动仍是这个语言。下拉框里每一项都用该语言自己的写法列出，所以无论当前界面是什么语言，都能认出该选哪一项。

2. 标注

2.1 进入标注界面

选中数据集后，右键缩略图选「标注」，或者直接双击缩略图。

2.2 画框

标注界面顶部工具栏有三种画法：

- **矩形** —— 拖鼠标画框
- **多边形** —— 依次点顶点，双击或回车结束（分割任务用）

- **文本** —— 先打开顶部的「文本标注」开关，再拖框，松开就弹输入框录文字，双击可改（OCR 任务用）

画文本框一律用**矩形**。倾斜的文字也用水平框盖住就行 —— 训练时多边形会被当作水平外接框处理，沿着轮廓描没有额外收益。

工具栏上还有「删除图像」，删掉当前这张。

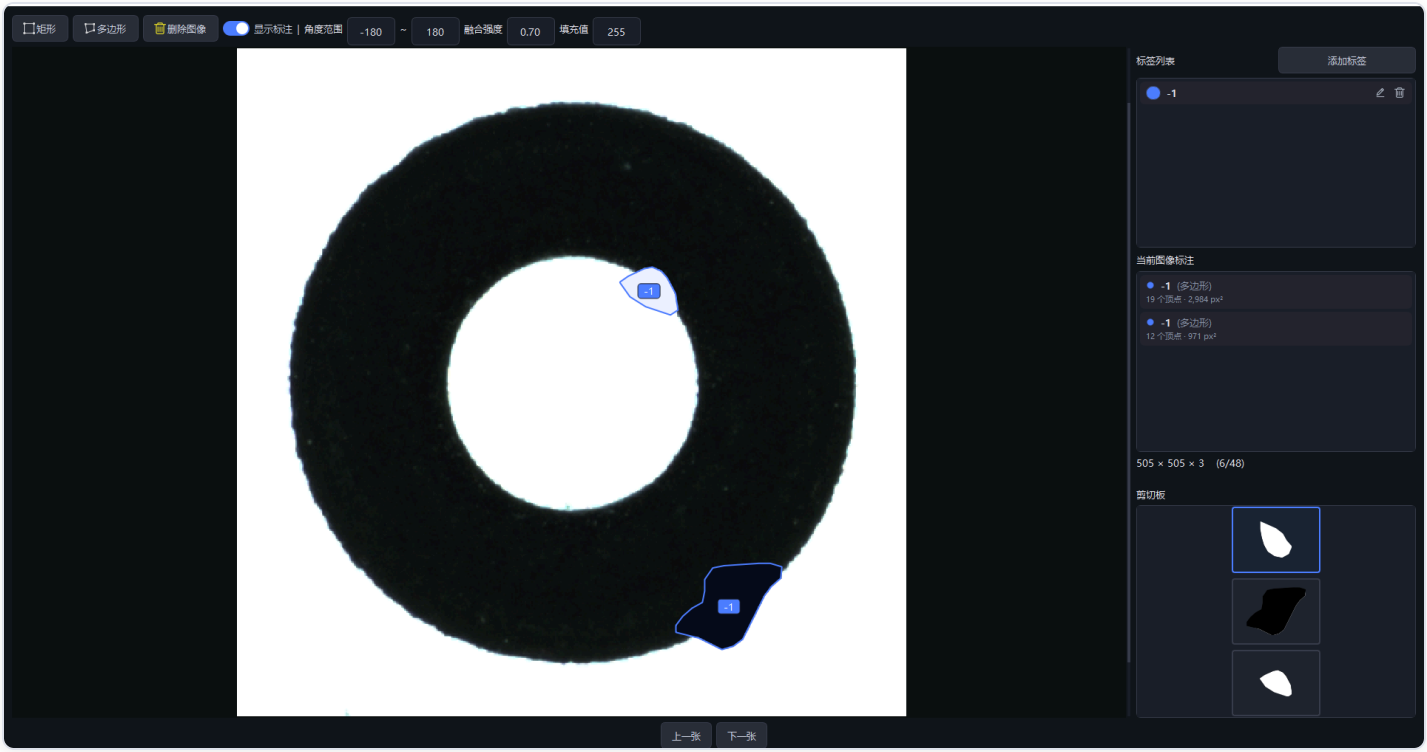
类别在右侧标签列表里选。

「显示标注」开关随时隐藏所有轮廓（框线、标签、顶点手柄），方便看原图。已经贴上去、还没落地的内容只藏轮廓、图案照常显示，所以关掉它正好能预览「贴完是什么效果」。

右下角**「标注信息」**列出本图每个标注对象的形状（矩形 / 多边形）、顶点数和面积；下面紧接着的**图像信息**显示当前图的宽 × 高 × 通道数（如 `505×505×3`）和序号。

再往下是**剪切板**：右键复制多边形后，复制区域的缩略图（`120 × 90`）会加进来，最多存 9 张，纵向排列、可滚动。最新一张默认选中、带蓝色边框；右键缩略图可以删掉这一张，或者清空全部。

剪切板里复制的内容可以**右键粘贴**回画布（默认带随机旋转），贴上去之后还能拖动、旋转。**格式刷**把徒手圈选的区域当模板，在任意位置刷出像素复制（可撤销）。**自动粘贴**对着「区域模板」面板里圈定的范围（多边形，只记区域、不成标注），每张图倒计时 3 秒随机贴 1~3 张，适合大批量重复贴图。



矩形 / 多边形标注、标签列表与右下角剪切板

2.3 标签管理

标注界面右侧是标签列表，上方有「添加标签」：

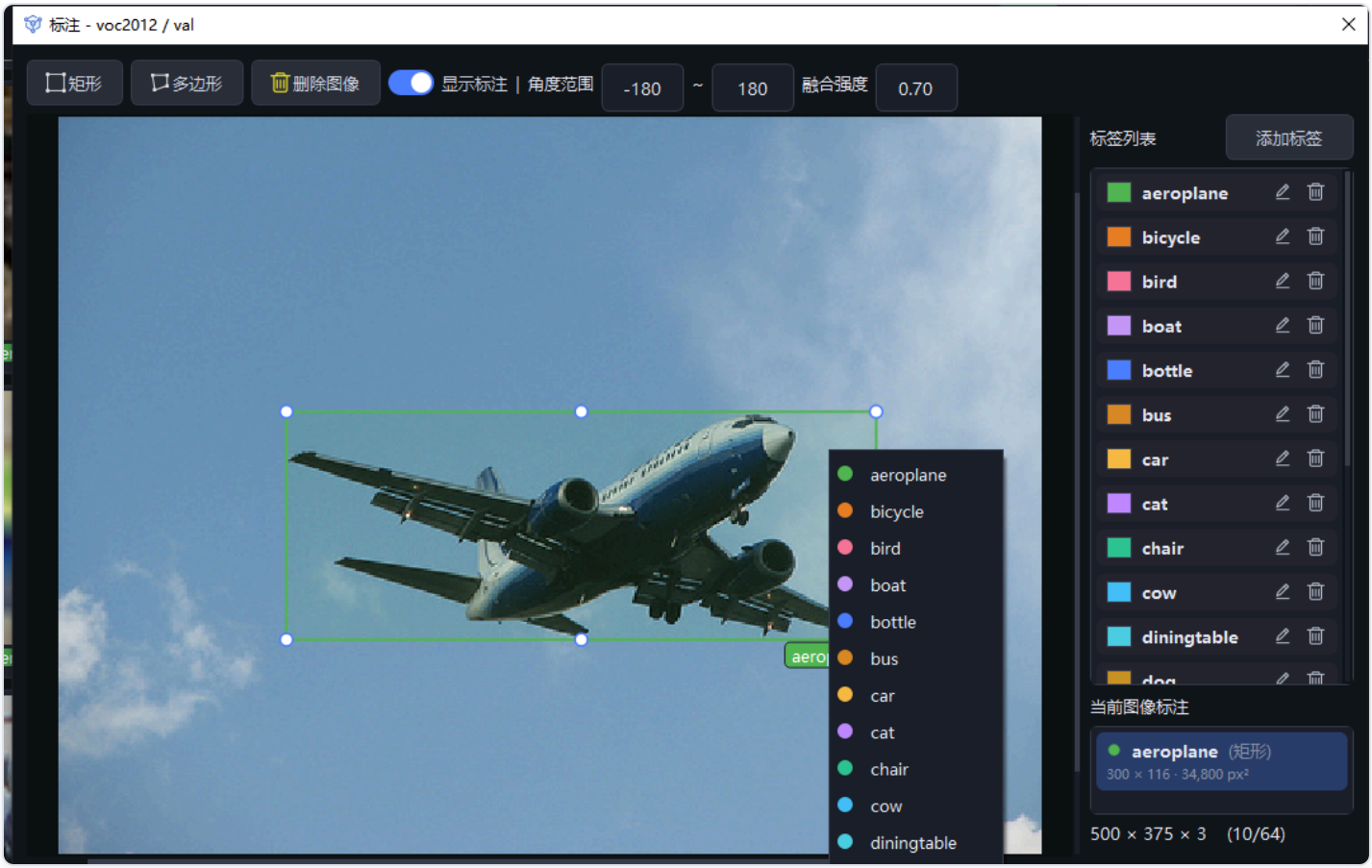
- **标签名称** —— 多个标签用逗号分隔，一次可以加好几个
- **导入** —— 从别的数据集把标签整批抄过来
- **颜色** —— 从预设色板里选，或者点「自定义」自己挑



添加或导入标签

2.4 改类别

改一个：点一下标注框，弹出类别下拉菜单，选新类别即可。右侧标签列表里的标签也能重命名、改色、删除。



在标注框上直接修改类别

批量改：在缩略图上右键 →「批量修改类别」，填入原类别和新类别，整个数据集里这一类的标注全部改名。



类别批量修改弹窗

2.5 复制粘贴标注对象

右键一个**多边形**标注，有两个操作：

- **复制** —— 把这块区域的像素存成模板，同时在右下角剪切板生成缩略图
- **填充** —— 把区域内的像素改成「填充值」那三个框对应的 RGB 颜色（每通道 `0 ~ 255`，默认 255）。常用在把缺陷抹成背景色

左键点选剪切板里的一张缩略图，再把鼠标移到画布上，会有一条多边形虚线跟着走 —— 形状是旋转后的真实轮廓，已经套用了这次摇出的角度。落点和朝向先看清楚，按 `Esc` 收掉虚线回到普通鼠标模式，再右键选「粘贴」落地。

在图像空白处右键选**「粘贴」**，就把剪切板里当前选中的那张贴到鼠标位置，同一张图或换一张图都行。粘贴前先在工具栏设好这几项：

- **角度范围** —— 粘贴时随机旋转的角度区间（默认 `-180 ~ 180`）。虚线预览用的就是这个区间里摇出来的角度，所见即所得
- **融合强度** —— `0 ~ 1`，默认 `0.70`。粘贴时自动做亮度补偿和边缘羽化，让贴上去的部分和背景自然衔接；设成 0 就是直接硬贴
- **填充值** —— R / G / B 三个输入框（每通道 `0 ~ 255`，默认 255），配合右键「填充」把区域抹成这个颜色
- `Ctrl` + `Z` —— 撤销最近一次粘贴或填充，像素恢复、随之新增的标注一起移除，可以连着撤

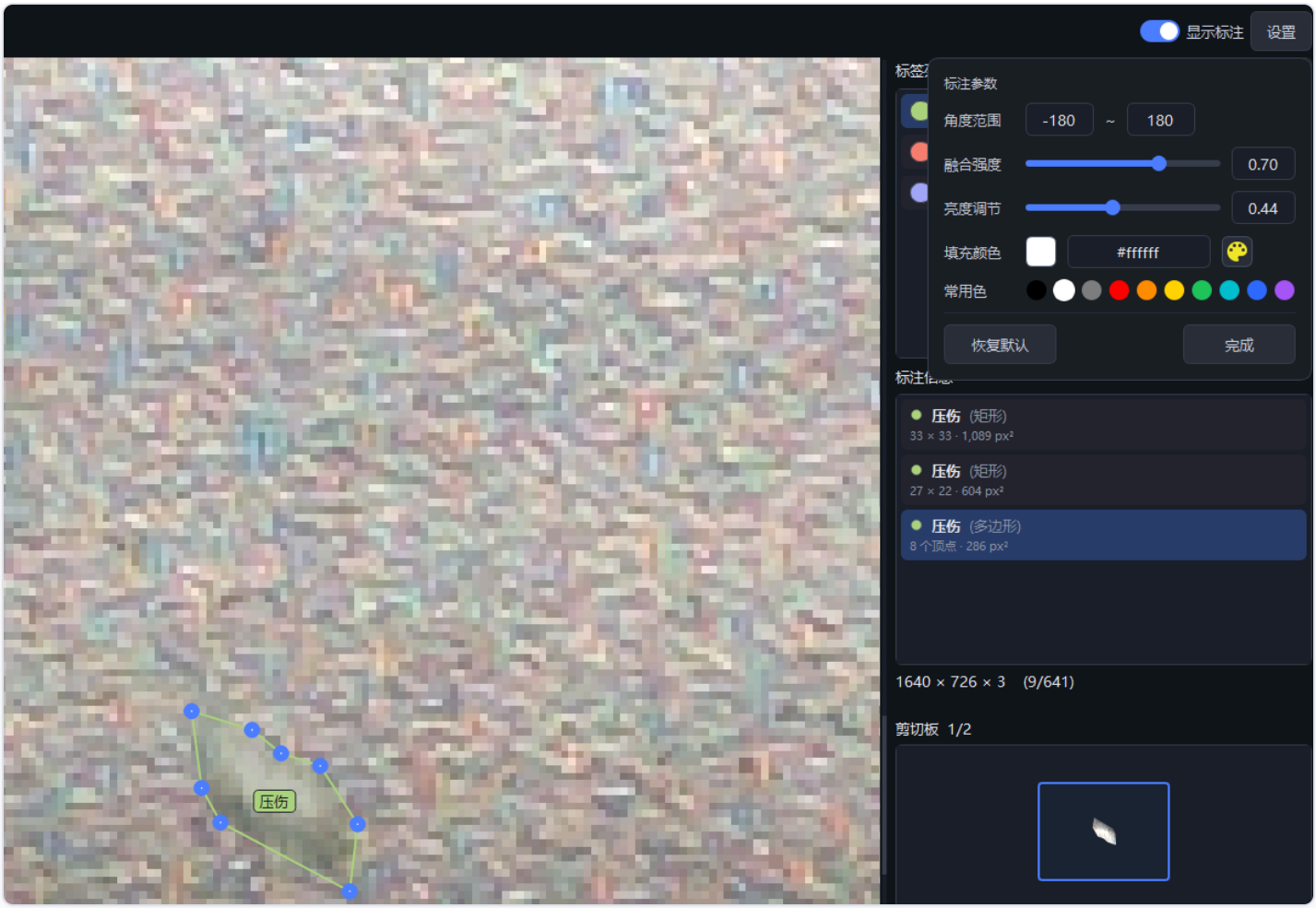


右键复制标注对象 → 剪切板缩略图，工具栏设置角度范围 / 融合强度 / 填充值

2.6 多边形区域亮度调节

点选一个**多边形**，在工具条「设置」里打开**亮度调节**，就能改这个多边形框内像素的明暗。拖滑块或者直接输数值，图像上实时预览。

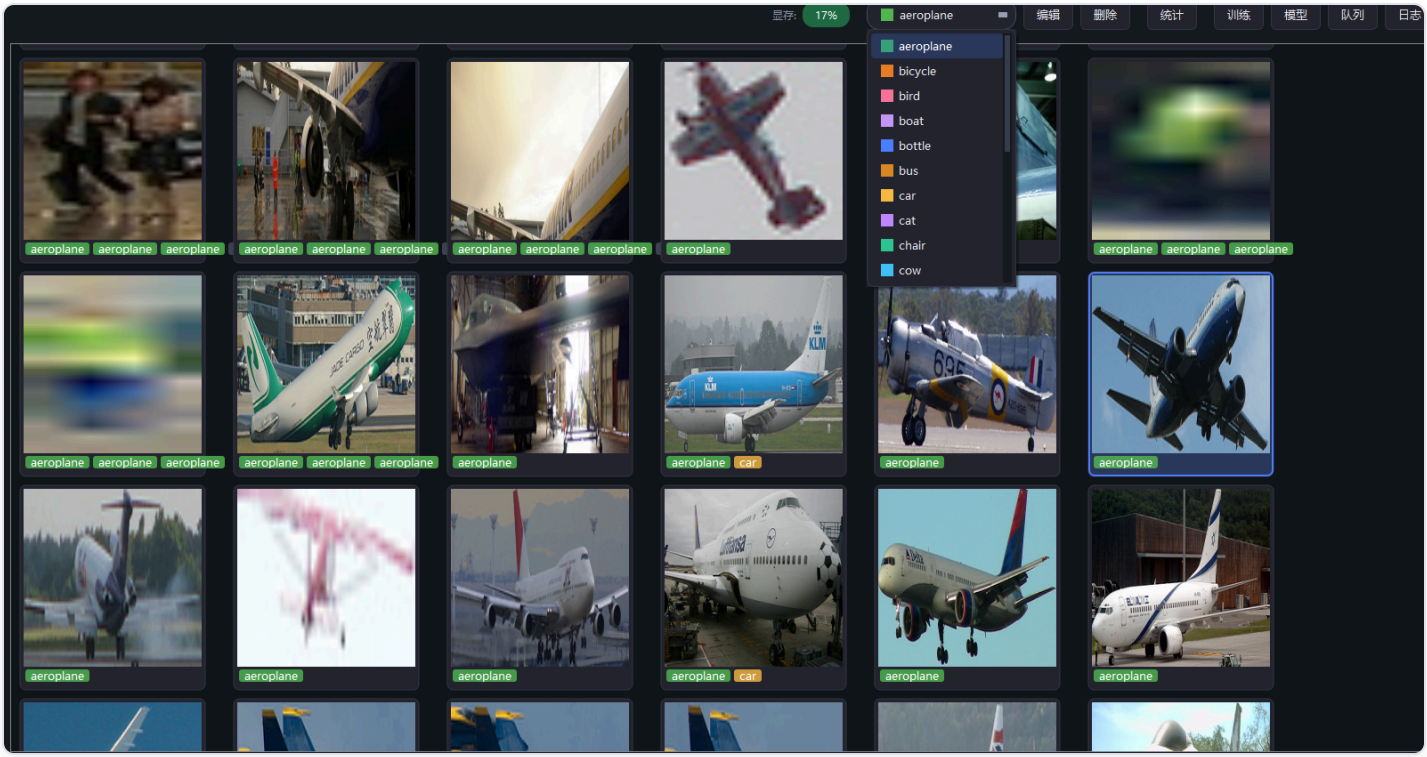
- **范围** —— `0 ~ 1`，默认 `0.50` 表示原样。往 1 走越亮（`1.00` 大约两倍亮度），往 0 走越暗（0 全黑）
- **只对选中的多边形生效。**矩形、或者什么都没选中的时候只给提示，不动像素。点另一个多边形，滑块会回到它自己的值（没调过的都是 0.50），不会把上一个的亮度带过去
- **什么时候写盘** —— 拖动过程只改内存、实时预览。按 `A` / `D` 切图或按 `Ctrl` + `S` 才写进图像文件
- **撤销** —— 一次调节只记一步，`Ctrl` + `Z` 回到调节前。同一次里来回拖多少下也只算一步



选中多边形后用「亮度调节」改框内像素明暗，图像上实时预览

2.7 按类别筛图

顶部工具栏的类别下拉框，选中某一类之后，图像区只显示含这个类别的图。想集中检查、补标某一类的时候很方便。



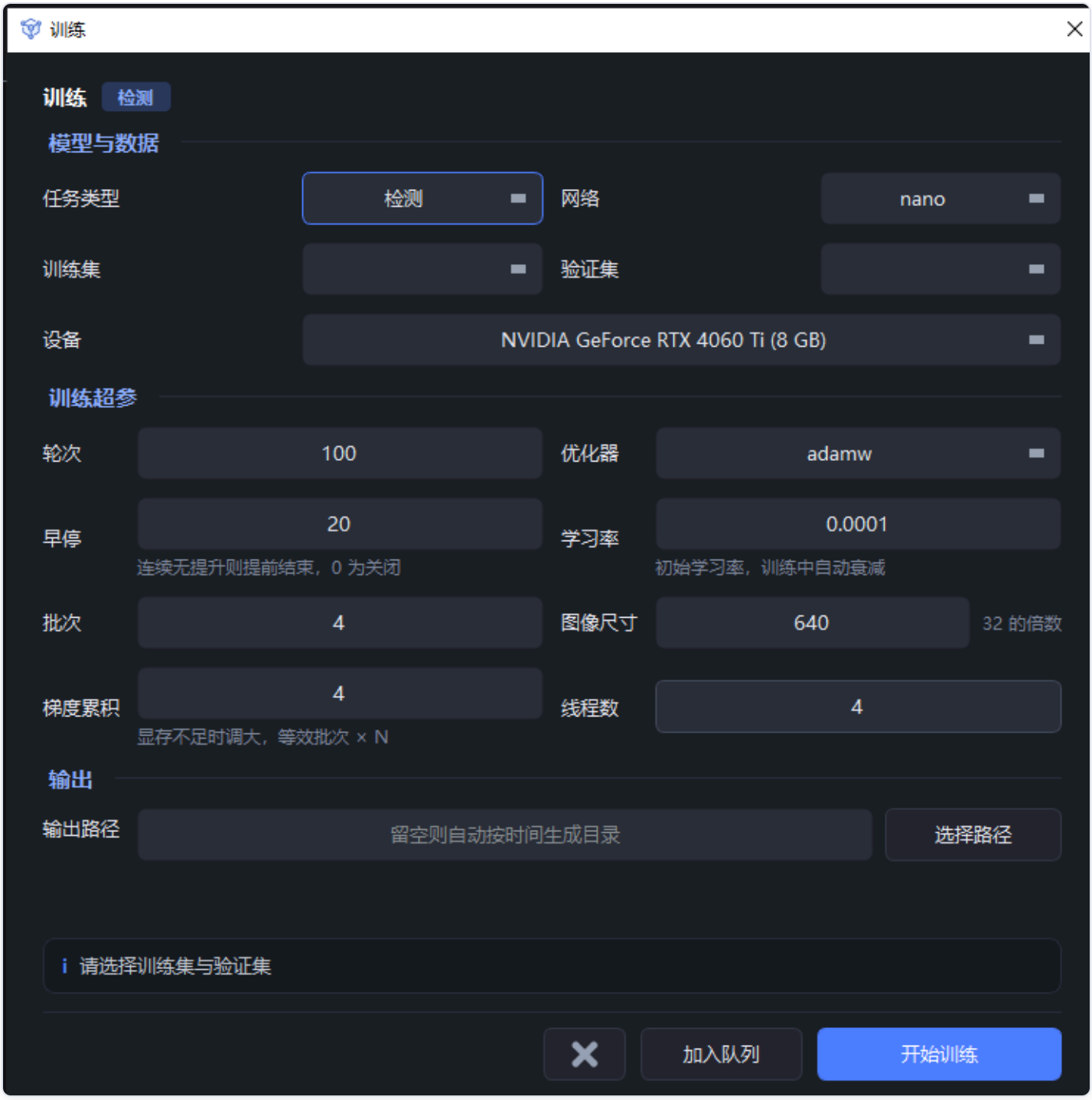
按类别筛选缩略图

3. 训练

3.1 配置训练参数

点首页或模型界面的「训练」，弹出参数配置：

- **任务类型** —— 检测 / 分割 / 分类 / 异常检测 / OCR，会跟着数据集格式自动切
- **训练集 / 验证集** —— 可以多选，也能跨项目组合
- **设备** —— 自动列出可用的 GPU
- **轮次 / 优化器 / 早停** —— 优化器检测和分割用 `adamw`、分类用 `sgd`；早停是连续 N 轮没有提升就提前结束，填 0 表示关闭
- **学习率** —— 训练过程中自动衰减
- **批次 / 图像尺寸 / 梯度累积 / 线程数** —— 图像尺寸必须是 32 的倍数；显存不够就调小批次、调大梯度累积，效果等同于放大批次
- **输出路径** —— 留空就按时间自动生成目录，也可以点「选择路径」指定



训练参数配置对话框

3.2 训练队列

参数配好之后，除了「开始训练」，还能点**「加入队列」**把任务先挂起来，接着配下一个。首页点「训练队列」打开队列窗口：

- 队列按顺序执行，检测 / 分割 / 分类 / 异常检测 / OCR 可以混排，跑完一项自动切下一项
- **上移 / 下移 / 移除** —— 调整执行顺序或删掉任务

- **编辑** —— 修改选中任务的参数
- **清理已完成** —— 把已经跑完的记录移出去
- **开始队列** —— 从头依次执行队列里的任务

> 队列适合晚上挂机：把多个数据集的任务一次配好，软件会自动全部跑完并分别保存模型。

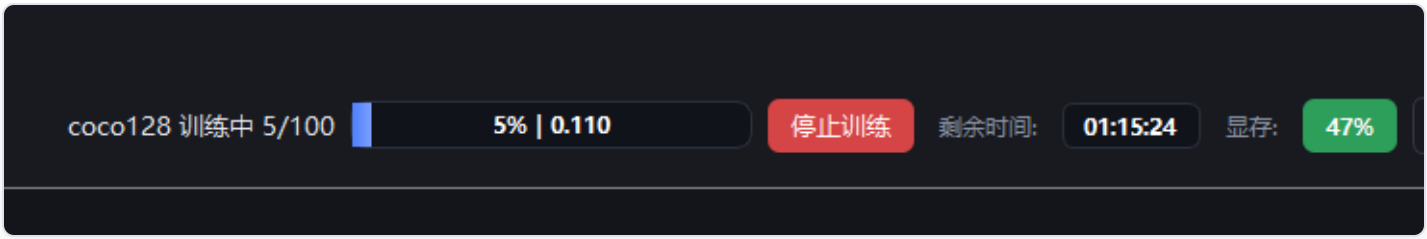


训练队列窗口

3.3 训练进度

点「开始训练」后回到首页，顶部出现进度条和控制条：

- **任务名 + 进度** —— 形如 `coco128 训练中 5/100`，进度条里显示当前轮的 loss
- **停止训练** —— 随时中止，已经跑完的部分会保存
- **剩余时间** —— 按已经用掉的时间外推估算
- **显存** —— GPU 显存占用率，实时刷新

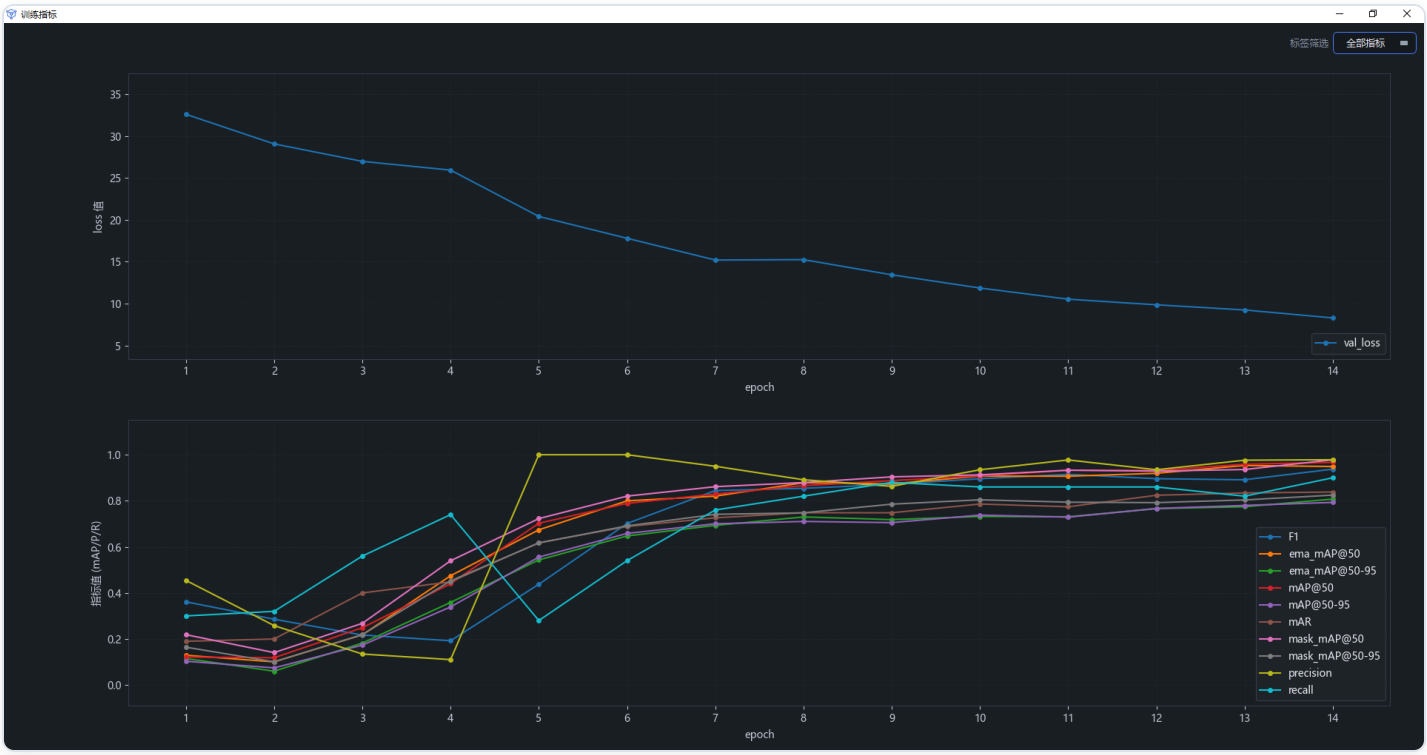


训练进度条与状态栏

3.4 训练曲线

在模型管理里选中一条训练记录，点右侧详情面板的**「查看完整指标」**，弹出训练曲线窗口：

- 上半部分是 **val loss** 曲线
- 下半部分是 mAP@50-95 / mAP@50 / AR / F1 / precision / recall 等多条曲线，分割任务还会多一条 mask mAP
- 右上角「标签筛选」下拉框，可以切换看全部指标或某个类别的曲线



训练指标曲线（val loss + 各项精度）

4. 模型管理

点首页「模型」打开模型管理：左边是训练记录列表，右边是详情面板。

4.1 列表与筛选

- **搜索框** —— 按项目 / 数据集 / 标签的关键字过滤
- **任务 / 状态下拉** —— 任务可选检测 / 分割 / 分类 / 异常检测 / 字符识别（早期的字符检测记录也并入此项筛选）；状态可选训练中 / 已完成 / 失败 / 已停止
- **仅看每个数据集最佳** —— 同一个数据集只留精度最高的那条
- 列表列：任务、数据集·标签、精度、训练时间、耗时、图像尺寸、操作
- **精度列带色条** —— 绿 ≥ 0.8、黄 0.5 ~ 0.8、红 < 0.5。失败或已停止的记录在这一列显示状态文字，鼠标停上去能看到失败原因
- 行内三个按钮：**测试**（打开模型测试）、**导出**、**删除**（连带清掉对应的训练记录）

4.2 详情面板

点任意一行，右侧显示这条记录的完整参数：任务·规模、状态、精度、训练 / 验证集、图像尺寸、轮数 / 早停、批大小、学习率、优化器、设备、标签、训练时间、耗时、模型路径。上方还有精度曲线的缩略图。

面板底部四个按钮：

- **查看完整指标** —— 打开训练曲线窗口（见「训练曲线」一节）
- **测试此模型** —— 用这个模型打开测试对话框（见「测试参数」一节）
- **按此配置重训** —— 把这条记录的全部训练参数带出来，重新开一次训练
- **打开模型目录** —— 在资源管理器里定位权重文件



模型管理：筛选条 + 训练记录列表 + 右侧详情面板

4.3 导出 ONNX

点「导出」选个目录，会生成一个「项目_时间戳」文件夹：

- **模型** `.onnx` —— 检测和分割由 RF-DETR 导出，分类与 OCR 由 torch 导出。命名是「项目_任务_图像尺寸_模型规模.onnx」
- `classes.txt` —— 类别 id 和名称的对照。OCR 的识别段是 `vocab.txt`，一行一个字符就是词表
- **评估报告 PDF** —— 自动在验证集上跑一遍评估，复用测试报告的模板，命名「..._评估报告.pdf」
- `examples/` —— C++ / C# / Python 调用 ONNX 做检测、分割、分类的示例代码。OCR 目前只有 Python 版

导出过程有进度提示。ONNX 转换需要装 `onnx` 和 `onnxsim`，没装会在弹窗里提示。

分类任务不生成评估报告 —— 漏检误检那套报告对分类不适用。OCR 的字符识别也一样，它的结论只有 CER。

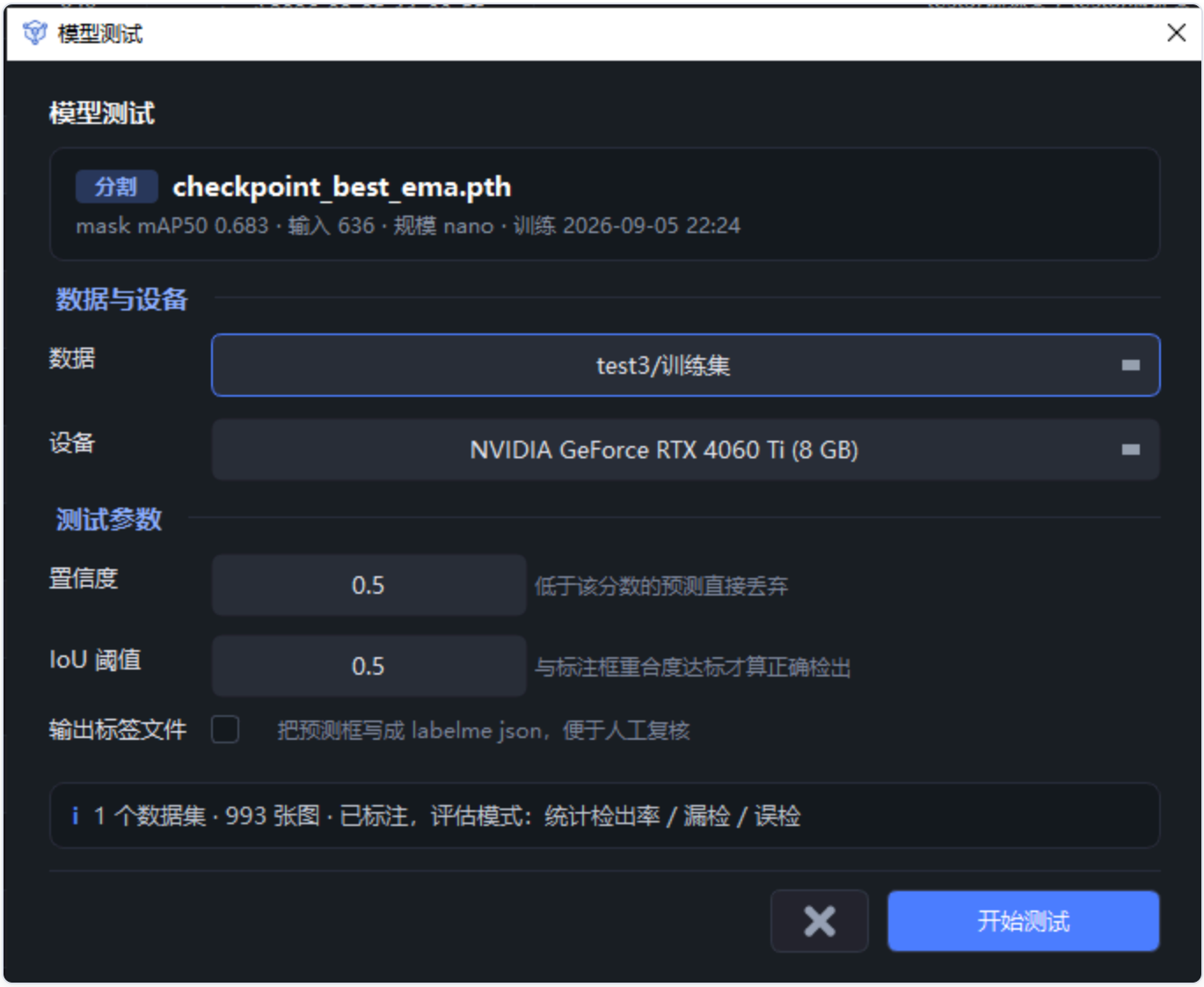
5. 测试与评估报告

5.1 测试参数

在模型管理里点行内的「测试」（或详情面板的「测试此模型」），弹出测试对话框。顶部显示模型信息：任务类型、权重文件、训练精度、输入尺寸、规模。下面是参数：

- **数据** —— 跨项目的所有数据集都能选
- **设备** —— 默认第一个可用的 GPU
- **置信度** —— 低于这个分数的预测直接丢掉
- **IoU 阈值** —— 预测框和标注框的重合度达标才算正确检出
- **输出标签文件** —— 勾上就把预测框写成 labelme json，方便人工复核
- **OCR** —— 字符识别拿标注框裁出的字条测试，报告字条总数、CER（字符错误率）和全对条数；纯图像（无标注）走推理，把识别出的文字写成标注 json 落回图像同目录。字符检测的推理 / 评估仅历史记录保留

> 数据集有标注时会自动进入**评估模式**，统计检出率 / 漏检 / 误检。（字符检测选推理模式时无视标注、直接把预测框写成 json，仅历史记录保留。）



模型测试参数配置

5.2 结果分析

测试跑完弹出结果分析窗口，分两个维度统计，用词一律是直白的**「正确检出 / 漏检 / 误检」**：

- **图像维度**（按张统计，检出 1 个就算检出）—— 测试张数、检出图像及检出率、未检出图像、有误检的图像
- **标注维度**（按标注框统计）—— 正确检出、漏检、误检、准确率
- **按类别表格** —— 每一类的标注数、正确检出、漏检、误检、检出率、准确率
- **字符识别** —— 不按框统计，报字条总数、CER（字符错误率）和全对条数
- **底部结论** —— 一句话点出整体是偏漏检还是偏误检，以及拖累最大的类别



测试结果分析窗口（图像维度 + 标注维度 + 按类别明细）

5.3 导出 PDF 报告

点右下角「导出 PDF 报告」，后台线程生成，内容包括：

- **总览页** —— 指标汇总表 + 标注分布图
- **漏检样本页 / 误检样本页** —— 按「每类抽取 N 张」展示缩略图（数量在左下角调），图上用红框标出漏检和误检的位置，并注明抽取比例
- **建议页** —— 针对各类错误给出改进方向

> 抽样只发生在出报告这一步，明细数据仍是全量，后面做难例挖掘不会丢数据。

6. OCR

OCR 只训练**字符识别**一段（docTR CRNN）：字符位置由标注框给出，不再训练检测模型。训练、测试、导出都只针对识别段，模型管理里一条记录对应一个识别模型；早期版本留下的字符检测记录仍可测试与导出。

6.1 标注文本

在标注界面打开顶部的**「文本标注」**开关，类别会自动固定成保留标签「文本」（不用先建标签）。像画矩形一样框住文字，松开鼠标就弹出输入框录内容；双击已有的文本框随时能改。

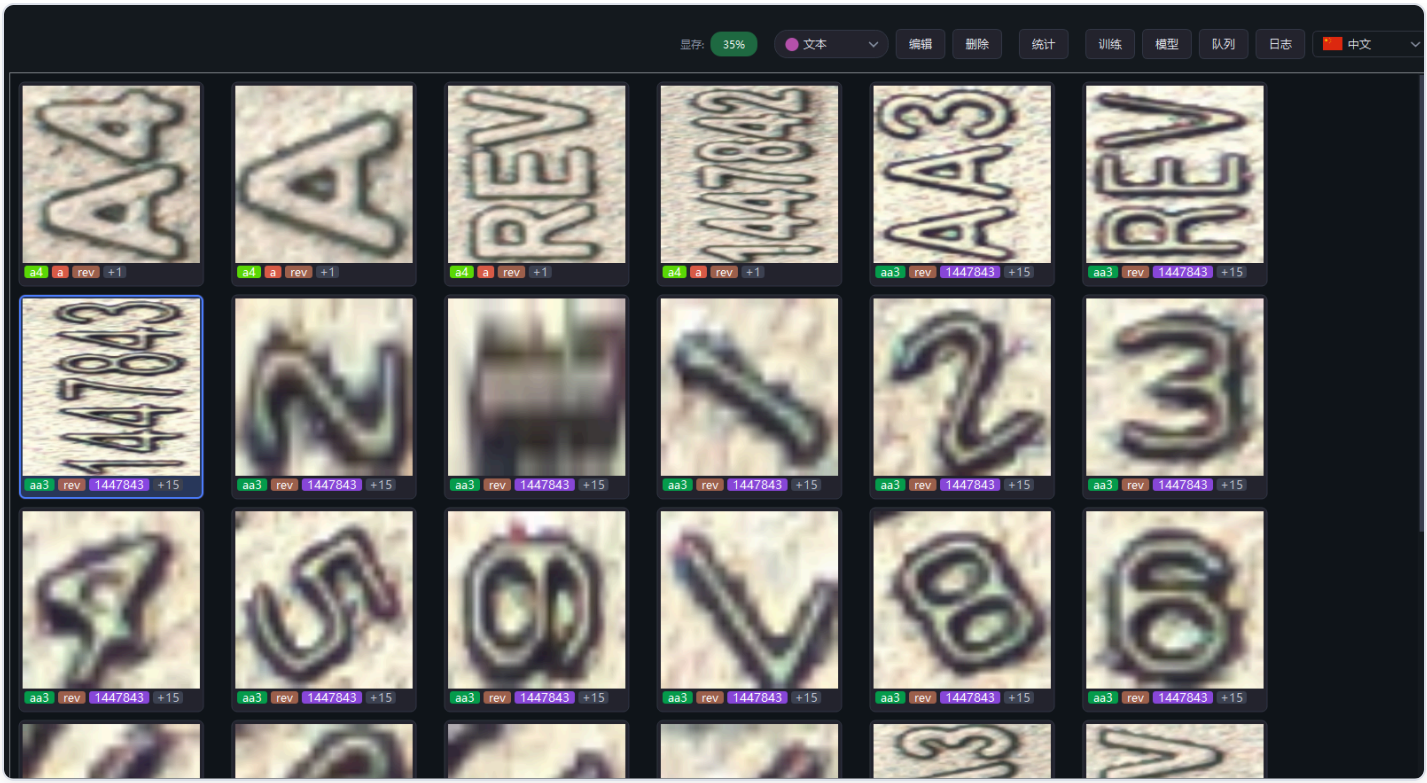
文本框一律用矩形画：倾斜的文字也用水平框盖住就行，训练时多边形会被当成水平外接框，描着轮廓画没有额外收益。

带文本框的 labelme json 导入时会逐图标记文本标注。往后只有含文本标注的数据集会出现在 OCR 的训练集下拉里。



打开「文本标注」开关后拖框，松开即弹出文字输入框；右侧「标注信息」列出每个文本框的形状、顶点数与面积

标完回到数据集视图，OCR 数据集里每个文本框会**单独裁成一张缩略图**，标签条上显示的是框里录的**文字内容**（超过 3 条折叠成 `+N`），不是类别名 —— 逐张翻过去就能核对内容录得对不对。上方类别筛选里只有「文本」这一项。



OCR 数据集：每个文本框单独成图，标签条显示所录文字内容（超出折叠成 +N）

6.2 训练

任务类型选 **OCR**。

图像尺寸这一项在字符识别下不参与训练（识别段固定 `32×768`，输入框已置灰并固定显示 768）。

点「开始训练」后直接训练字符识别段，队列里只排这一项。



OCR 的训练参数；型号即档位 nano ~ large，架构一栏由算法自动决定、置灰不可改

> 识别段的词表直接取标注里出现过的字符。所以标注文本要覆盖现场实际会遇到的字符集 —— 现场有、标注里没出现过的字符，模型认不出来。

6.3 测试

- **字符识别 · 评估** —— 数据集有标注时自动进入，拿标注框裁出的字条比对，报告字条总数、CER（字符错误率）和全对条数
- **字符识别 · 推理** —— 纯图像（无标注）时进入，把识别出的文字写成 labelme json 落回图像同目录，首页重载即可核对

- **字符检测 · 推理 / 评估** —— 仅早期版本留下的检测段记录可用：推理把预测框写成 json，评估统计正确检出 / 漏检 / 误检和按类 P/R

6.4 导出

导出一份 ONNX：`项目_字符识别_尺寸_规模.onnx`，外加一个 `vocab.txt` 词表（一行一个字符）。早期版本的字符检测记录仍可导出检测段 ONNX。

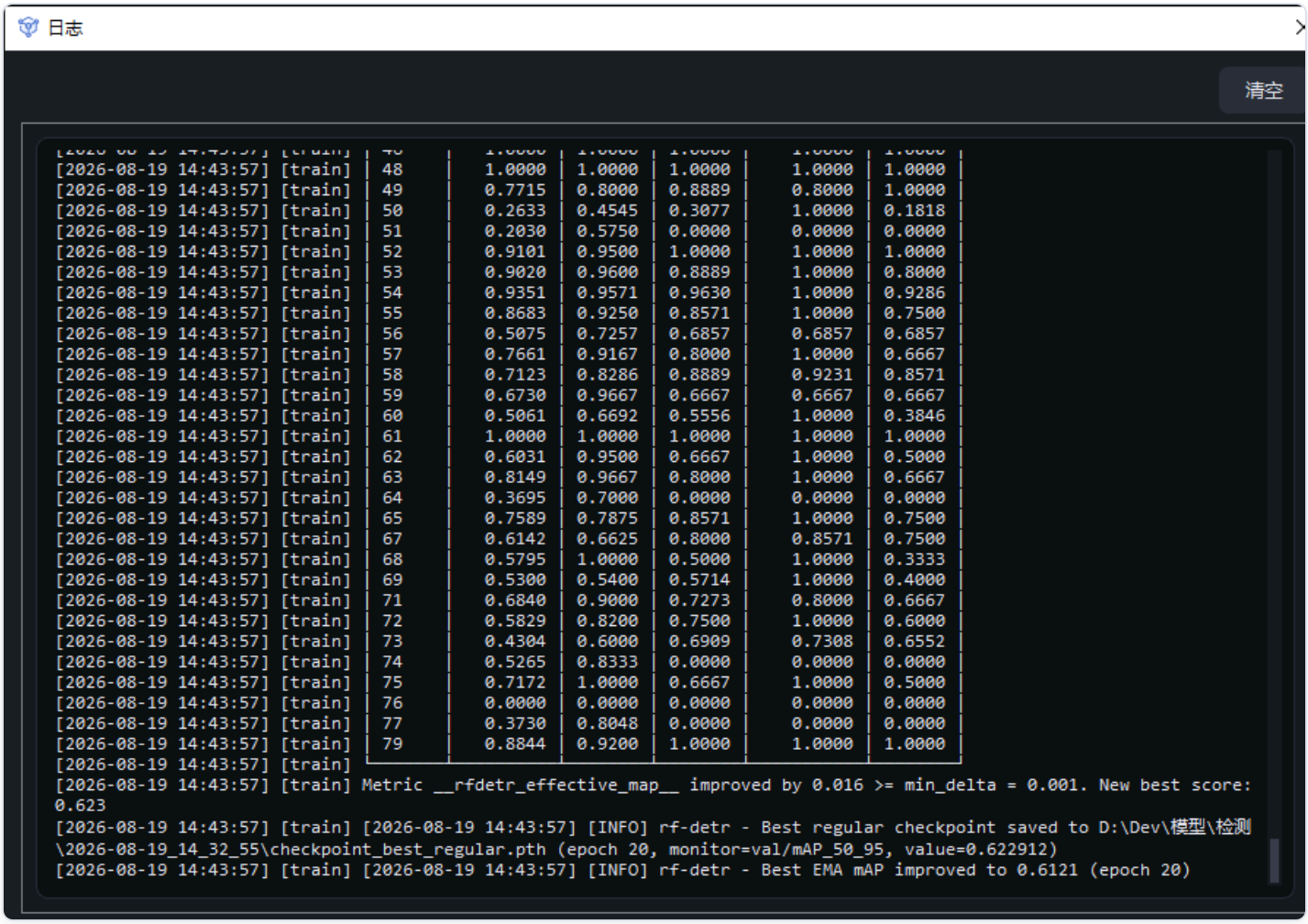
字符识别只有 CER 一个结论，不生成评估报告。

调用示例在导出目录的 `examples/python/ocr.py`。

7. 日志

点首页顶部的「日志」，打开日志窗口，实时滚动显示：

- 软件启动 / 退出
- 项目和数据集的创建、重命名、删除、导入 / 导出
- 标签的创建、重命名、删除
- 训练的开始、进行中（`[train]` 前缀转发子进程输出）、指标更新、模型保存、手动停止
- 测试的启动、进行中（`[test]` 前缀）、完成、失败


日志
清空
[2026-08-19 14:43:57] [train] | 48 | 1.0000 | 1.0000 | 1.0000 | 1.0000 | 1.0000 |
[2026-08-19 14:43:57] [train] | 49 | 0.7715 | 0.8000 | 0.8889 | 0.8000 | 1.0000 |
[2026-08-19 14:43:57] [train] | 50 | 0.2633 | 0.4545 | 0.3077 | 1.0000 | 0.1818 |
[2026-08-19 14:43:57] [train] | 51 | 0.2030 | 0.5750 | 0.0000 | 0.0000 | 0.0000 |
[2026-08-19 14:43:57] [train] | 52 | 0.9101 | 0.9500 | 1.0000 | 1.0000 | 1.0000 |
[2026-08-19 14:43:57] [train] | 53 | 0.9020 | 0.9600 | 0.8889 | 1.0000 | 0.8000 |
[2026-08-19 14:43:57] [train] | 54 | 0.9351 | 0.9571 | 0.9630 | 1.0000 | 0.9286 |
[2026-08-19 14:43:57] [train] | 55 | 0.8683 | 0.9250 | 0.8571 | 1.0000 | 0.7500 |
[2026-08-19 14:43:57] [train] | 56 | 0.5075 | 0.7257 | 0.6857 | 0.6857 | 0.6857 |
[2026-08-19 14:43:57] [train] | 57 | 0.7661 | 0.9167 | 0.8000 | 1.0000 | 0.6667 |
[2026-08-19 14:43:57] [train] | 58 | 0.7123 | 0.8286 | 0.8889 | 0.9231 | 0.8571 |
[2026-08-19 14:43:57] [train] | 59 | 0.6730 | 0.9667 | 0.6667 | 0.6667 | 0.6667 |
[2026-08-19 14:43:57] [train] | 60 | 0.5061 | 0.6692 | 0.5556 | 1.0000 | 0.3846 |
[2026-08-19 14:43:57] [train] | 61 | 1.0000 | 1.0000 | 1.0000 | 1.0000 | 1.0000 |
[2026-08-19 14:43:57] [train] | 62 | 0.6031 | 0.9500 | 0.6667 | 1.0000 | 0.5000 |
[2026-08-19 14:43:57] [train] | 63 | 0.8149 | 0.9667 | 0.8000 | 1.0000 | 0.6667 |
[2026-08-19 14:43:57] [train] | 64 | 0.3695 | 0.7000 | 0.0000 | 0.0000 | 0.0000 |
[2026-08-19 14:43:57] [train] | 65 | 0.7589 | 0.7875 | 0.8571 | 1.0000 | 0.7500 |
[2026-08-19 14:43:57] [train] | 67 | 0.6142 | 0.6625 | 0.8000 | 0.8571 | 0.7500 |
[2026-08-19 14:43:57] [train] | 68 | 0.5795 | 1.0000 | 0.5000 | 1.0000 | 0.3333 |
[2026-08-19 14:43:57] [train] | 69 | 0.5300 | 0.5400 | 0.5714 | 1.0000 | 0.4000 |
[2026-08-19 14:43:57] [train] | 71 | 0.6840 | 0.9000 | 0.7273 | 0.8000 | 0.6667 |
[2026-08-19 14:43:57] [train] | 72 | 0.5829 | 0.8200 | 0.7500 | 1.0000 | 0.6000 |
[2026-08-19 14:43:57] [train] | 73 | 0.4304 | 0.6000 | 0.6909 | 0.7308 | 0.6552 |
[2026-08-19 14:43:57] [train] | 74 | 0.5265 | 0.8333 | 0.0000 | 0.0000 | 0.0000 |
[2026-08-19 14:43:57] [train] | 75 | 0.7172 | 1.0000 | 0.6667 | 1.0000 | 0.5000 |
[2026-08-19 14:43:57] [train] | 76 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |
[2026-08-19 14:43:57] [train] | 77 | 0.3730 | 0.8048 | 0.0000 | 0.0000 | 0.0000 |
[2026-08-19 14:43:57] [train] | 79 | 0.8844 | 0.9200 | 1.0000 | 1.0000 | 1.0000 |
[2026-08-19 14:43:57] [train]
[2026-08-19 14:43:57] [train] Metric __rfdetr_effective_map__ improved by 0.016 >= min_delta = 0.001. New best score: 0.623
[2026-08-19 14:43:57] [train] [2026-08-19 14:43:57] [INFO] rf-detr - Best regular checkpoint saved to D:\Dev\模型\检测\2026-08-19_14_32_55\checkpoint_best_regular.pth (epoch 20, monitor=val/mAP_50_95, value=0.622912)
[2026-08-19 14:43:57] [train] [2026-08-19 14:43:57] [INFO] rf-detr - Best EMA mAP improved to 0.6121 (epoch 20)

日志窗口